

Rapport de Projet : Régression Logistique pour la Classification de Chiffres Manuscrits

Lilian NOACCO
Basile LE THIEC
2A Alt IR

1 Introduction

Ce rapport détaille l'implémentation d'un classifieur d'images *from scratch* en utilisant la régression logistique multi-classes. L'objectif est de classer les images de chiffres manuscrits de la base de données "digits" fournie par scikit-learn. Ce document couvre la théorie mathématique, la mise en œuvre, la comparaison avec l'implémentation de référence de scikit-learn, et une analyse des résultats obtenus.

2 Théorie Mathématique

2.1 Régression Logistique Multi-classes

La régression logistique est un modèle linéaire utilisé pour la classification. Pour un problème à K classes, nous cherchons à modéliser la probabilité qu'une entrée $\mathbf{x}$ appartienne à la classe k : $P(y = k|\mathbf{x})$.

Le score (ou logit) pour chaque classe k est calculé comme une fonction linéaire de $\mathbf{x}$:

$$z_k = \mathbf{x} \cdot \mathbf{W}_k^T + b_k \quad (1)$$

Où :

- $\mathbf{W}_k$ est le vecteur de poids pour la classe k .
- b_k est le biais pour la classe k .

2.2 Fonction Softmax

Pour convertir ces scores en probabilités, nous utilisons la fonction Softmax, qui garantit que la somme des probabilités pour toutes les classes est égale à 1.

$$P(y = k|\mathbf{x}) = \text{softmax}(z)_k = \frac{\exp(z_k)}{\sum_{j=1}^K \exp(z_j)} \quad (2)$$

Pour des raisons de stabilité numérique, on soustrait généralement le score maximum avant de passer à l'exponentielle : $\exp(z_k - \max(\mathbf{z}))$.

2.3 Descente de Gradient et Fonction de Coût

L'objectif est de trouver les paramètres $\mathbf{W}$ et $\mathbf{b}$ qui minimisent une fonction de coût. Pour la régression logistique multi-classes, on utilise la fonction de coût d'entropie croisée (Cross-Entropy Loss) :

$$J(\mathbf{W}, \mathbf{b}) = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_{i,k} \log(p_{i,k}) \quad (3)$$

Où N est le nombre d'échantillons, $y_{i,k}$ est 1 si l'échantillon i appartient à la classe k (encodage one-hot), et $p_{i,k}$ est la probabilité prédite.

Nous utilisons la descente de gradient pour minimiser cette fonction. Les gradients sont :

$$\nabla_{\mathbf{W}_k} J = \frac{1}{N} \sum_{i=1}^N (p_{i,k} - y_{i,k}) \mathbf{x}_i \quad (4)$$

$$\nabla_{b_k} J = \frac{1}{N} \sum_{i=1}^N (p_{i,k} - y_{i,k}) \quad (5)$$

Les poids sont mis à jour comme suit, où η est le taux d'apprentissage :

$$\mathbf{W}_k := \mathbf{W}_k - \eta \nabla_{\mathbf{W}_k} J \quad (6)$$

$$b_k := b_k - \eta \nabla_{b_k} J \quad (7)$$

2.4 Régularisation L2 (Bonus)

Pour prévenir le surapprentissage, nous avons ajouté une régularisation L2 (Ridge) qui modifie la fonction de coût :

$$J_{\text{reg}}(\mathbf{W}, \mathbf{b}) = J(\mathbf{W}, \mathbf{b}) + \frac{\alpha}{2} \sum_{k=1}^K \|\mathbf{W}_k\|^2 \quad (8)$$

Le gradient pour les poids est alors modifié :

$$\nabla_{\mathbf{W}_k} J_{\text{reg}} = \nabla_{\mathbf{W}_k} J + \alpha \mathbf{W}_k \quad (9)$$

3 Implémentation et Résultats

Le script `projet-v1.py` met en œuvre ce qui suit :

- **Traitement des données** : Chargement du jeu de données, encodage one-hot et division en ensembles d'entraînement et de test.
- **Implémentation from scratch** : La classe `CustomLogisticRegression`.
- **Comparaison** : Le modèle custom est comparé à `LogisticRegression` de scikit-learn.
- **Bonus** : Une version avec régularisation L2 a été implémentée.

3.1 Résultats Obtenus

- **Modèle Custom (sans régularisation)** : Accuracy = 96.94%
- **Modèle Scikit-learn** : Accuracy = 97.50%
- **Modèle Custom (avec régularisation L2)** : Accuracy = 97.50%

L'ajout de la régularisation L2 a permis à notre modèle d'égaliser les performances de l'implémentation de scikit-learn.

4 Analyse et Réflexion

4.1 Interprétation des Coefficients

Les poids ($\mathbf{W}$) appris par le modèle peuvent être visualisés comme des images. Ces images représentent les "templates" que le modèle a appris pour chaque chiffre.

4.2 Analyse des Erreurs

L'analyse des images mal classées montre souvent des chiffres ambigus ou mal formés, ce qui explique les erreurs du modèle linéaire.

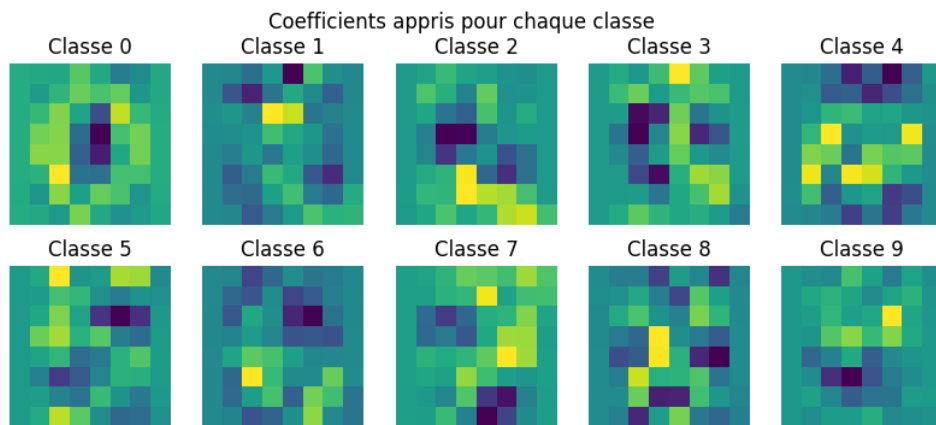


Figure 1: Coefficients appris pour chaque classe (visualisation des poids comme images 8x8).

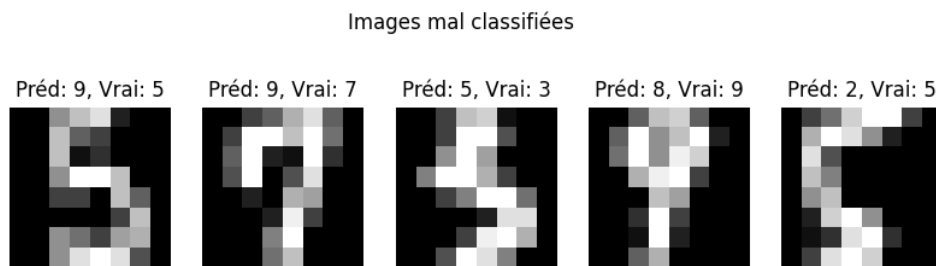


Figure 2: Exemples d'images mal classifiées (prédiction vs étiquette réelle).

4.3 Prise de Recul

- **Difficultés** : Assurer la stabilité numérique de la fonction softmax.
- **Limites** : La régression logistique est un modèle simple. Des modèles plus complexes comme les réseaux de neurones convolutifs (CNN) sont généralement plus performants pour cette tâche.
- **Paramètres** : Le choix du `learning_rate` et de `max_iter` est crucial et nécessite une validation croisée pour un réglage optimal.

5 Conclusion

Ce projet a permis de mettre en pratique la théorie de la régression logistique. L'implémentation *from scratch* a donné d'excellents résultats, et l'ajout de la régularisation a permis de combler l'écart de performance avec une librairie de référence.